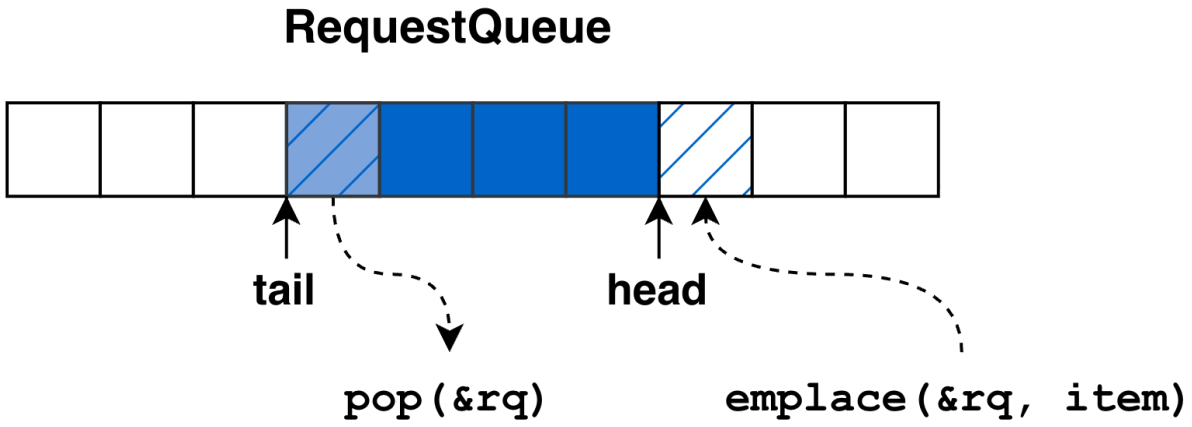
Kaiyuan Zhang, 811597925

(1) Method

i) Data Structure

I defined a new struct, RequestQueue, as the buffer to store the request message.`emplace() pushes the requested content, pop() pops the content.



ii) Synchronization

(a) **Global Variables**

```
RequestQueue rq;                        // buffer
sem_t sem_empty, sem_full;
pthread_mutex_t mutex;
```

(b) **Listener(producer) Thread**

```
sem_wait(&sem_empty);               // if full, wait
pthread_mutex_lock(&mutex);         // only this thread access
emplace(&rq, s);                    // do
pthread_mutex_unlock(&mutex);       // unlock buffer access
sem_post(&sem_full);                // sem_full++
```

(c) **Worker(consumer) Thread**

```
sem_wait(&sem_full);            // if empty wait, wait
pthread_mutex_lock(&mutex);     // only this thread access rq
fd = pop(&rq);                  // do
pthread_mutex_unlock(&mutex);   // unlock buffer access
sem_post(&sem_empty);           // sem_empty++
```

iii) Concurrency Problems

(a) Race condition

A race condition could occur if two worker threads update `rq.head` simultaneously. For example, suppose the current value of `rq.head` is `h`. Worker A reads `rq.head` into a register but is interrupted before storing the updated value. Worker B then reads the same value `h`, retrieves `buffer[h]`, and updates `rq.head` to `(h+1)%MAX_REQUEST`. When worker A resumes, it also retrieves buffer[h] and stores the same updated value. As a result, the same `fd` could be processed twice, while another request could be skipped.

To prevent this race condition, each worker first calls `sem_wait(&sem_full)` to wait until a request is available. It then calls `pthread_mutex_lock(&mutex)` before accessing the shared buffer and updating `rq.head`. Therefore, only one worker can execute `pop()` at a time.

The listener is the single producer. It calls `sem_wait(&sem_empty)` to wait until an empty buffer slot is available and then acquires the same mutex before inserting the file descriptor and updating `rq.tail`.

Thus, `sem_empty` and `sem_full` prevent listeners or the worker doing job while the buffer is full or empty. The `mutex` prevents race conditions on the shared queue.

(b) Deadlock

Case:

If in the listener thread, we do

`lock(mutex)-wait(empty)-emplace()-post(full)-unlock(mutex)`

can lead to “circular wait”. When buffer is full:

1. Listener lock `mutex`
2. Listener waits at `sem_wait(&sem_empty)` , because no empty block in buffer
3. Worker needs `mutex` to execute `pop()`, but cannot get mutex
4. Work and listen wait each other

My Solution:

I call `sem_wait(&sem_empty)` before acquiring the mutex:

`wait(empty)-lock(mutex)-emplace()-unlock(mutex)-post(full)`

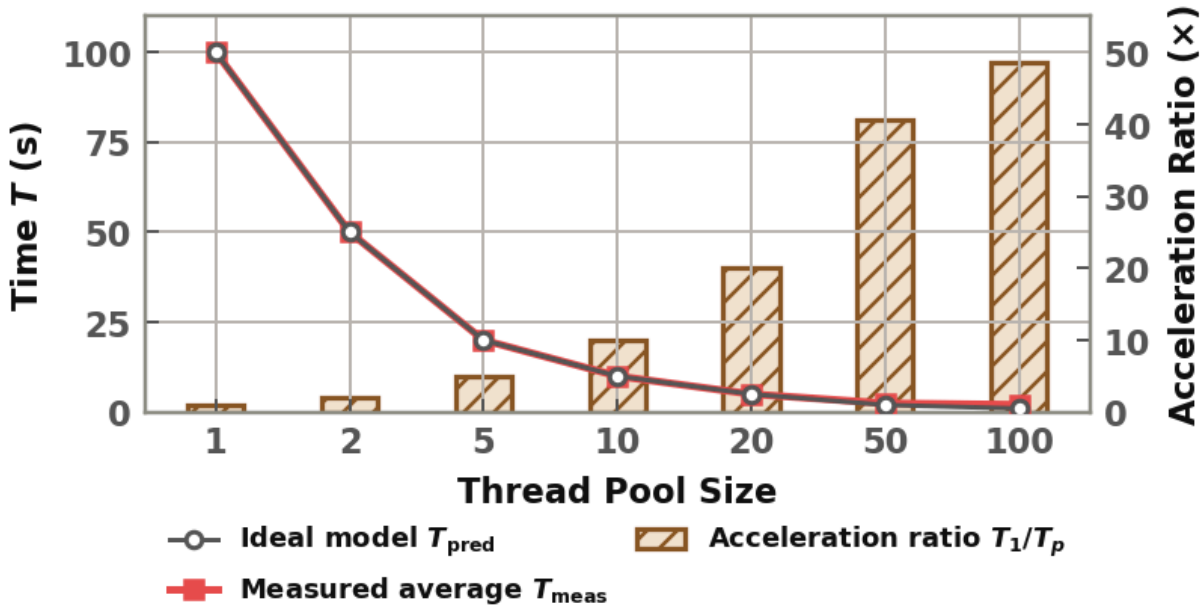
Therefore, a thread waiting for the buffer state to change does not hold the `mutex`. In addition, a thread holding the `mutex` never waits on another semaphore. This breaks the circular-wait condition, so a deadlock cannot occur during normal queue operation.

(2) Evaluation

Testbed: Ubuntu 26.04, AMD Ryzen 9700X

i) Acceleration

n_request: 100; n_threadpool: 1 - 100;



Acceleration ratio can reach 48.5x when n_threadpool=100.

ii) Race condition

I tested the implementation using ThreadSanitizer. No race was reported for `RequestQueue`, `rq.head`, or `rq.tail`. However, ThreadSanitizer detected races in the supplied process() implementation caused by the non-reentrant functions strtok() and gmtime().

iii) Deadlock

To test for deadlock, I started the server with 10 worker threads and issued 200 simultaneous requests. Because the queue capacity was 100, this workload forced the listener to wait on `sem_empty`. All 200 requests completed successfully. Therefore, no deadlock was observed, and the server remained responsive after the buffer-full condition.

Appendix

A. Shared Buffer Data Structure

```
typedef struct RequestQueue {
   int buffer[MAX_REQUEST];
   int head;
   int tail;
} RequestQueue;

void emplace(RequestQueue *q, int fd) {
   q->buffer[q->tail] = fd;
   q->tail = (q->tail + 1) % MAX_REQUEST;
}
int pop(RequestQueue *q) {
   int r = q->buffer[q->head];
   q->head = (q->head + 1) % MAX_REQUEST;
   return r;
}
```